NED UNIVERSITY OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND IT

COMPLEX COMPUTING PROBLEM (CCP)

REPORT

Course Code: **CT-351** Course Title: **Knowledge Representation & Reasoning**

Project Title:	**CyberGuard** A Knowledge Representation & Rule-Based Reasoning System for Cybersecurity Threat Detection and Access Control
Selected Domain:	Cybersecurity: Threat Detection and Access Control in a University Network
Section:	Artificial Intelligence
Submission Date:	9 November 2026

GROUP MEMBERS

S. No.	Student Name	Roll Number	Assigned Contribution
1	Umair Hasan	AI-23018	Backward chaining, decision logic, testing, report integration and references (Sections 6.2, 7.2, 8.1 and 10)
2	Abrar Shakeel Ghoury	AI-23019	Domain analysis, frames, script, semantic network and diagrams (Sections 2, 3 and 7.1)
3	Amna Amjad	AI-23031	Introduction, Horn clause knowledge base and forward chaining (Sections 1, 5 and 6.1)
4	Mirza Abdul Raffay Baig	AI-23033	First-Order Logic, Description Logic axioms, limitations analysis and conclusion (Sections 4, 8.2 and 9)

INSTRUCTOR	CLO	PLO
Ms. Dureshahwar Waseem	**CLO-3 (Design/Analysis)**	**PLO-10 (Lifelong Learning)**

REPORT CONTENTS

CT-351 | Knowledge Representation & Reasoning

1. Introduction 3

2. Domain Analysis 3

3. Knowledge Representation 5

3.1 Frames 6

3.2 Semantic Networks 9

4. Formal Logic Representation 12

4.1 First-Order Logic (FOL) 12

4.2 Description Logic (DL) 14

5. Horn Clause Knowledge Base 17

6. Inference and Reasoning 19

6.1 Forward Chaining 19

6.2 Backward Chaining 20

7. Implementation 24

8. Results and Discussion 30

9. Conclusion 34

10. References 34

11. Team Participation 35

1. INTRODUCTION

1.1 Selected Domain and Background

The selected domain is **cybersecurity**, more precisely **threat detection and access control in a university network**. A university network is used by students, faculty, administrators and IT staff, who log in from many devices and open resources of very different sensitivity, from a public course page to the grades database and the administrator panel. Whenever someone asks for a resource, the system must decide whether the request is normal, suspicious or dangerous. In practice this decision depends on who the person is, which device is used, how the login went and how sensitive the resource is.

CyberGuard is a rule-based Knowledge Representation and Reasoning (KRR) system [2] for this problem. It represents users, devices, resources, login events, permissions and threats as structured knowledge using **Frames** and a **Semantic Network**. It formalises the security policy in **First-Order Logic (FOL)** and **Description Logic (DL)**, converts the policy into **Horn clauses**, and applies **Forward Chaining** and **Backward Chaining** to decide for every access request whether it should be **ALLOWED**, **FLAGGED** or **DENIED**. Every decision comes with an explanation of how it was derived. The reasoning engine (unification, forward chaining and backward chaining) is written in Python by the team and does not use an external reasoning library.

1.2 Problem Statement and Objectives

A security decision is not stored anywhere in the data; it is the end of a chain of derived conclusions. For the scenario of the project proposal the chain is:

Multiple failed attempts → Brute Force → Suspicious Login → High-Risk Access → Access Denied

The problem is therefore to build a system that starts from basic facts (roles, devices, failed login attempts, requested resources), derives the intermediate security concepts by logical rules, reaches an ALLOW, FLAG or DENY decision, and explains why. The main objectives of the project, which follow the CCP guidelines of the course [5], are:

- represent the domain with frames (slots, facets, default values, inheritance, a demon and a script) and a semantic network (is-a, has-a, part-of links);
- formalise the facts and rules in FOL and DL (subsumption, membership and disjointness) and analyse the trade-off between expressive power and tractability;
- convert the rules to Horn clauses and run step-by-step data-driven (forward) and goal-driven (backward) inference;
- produce an ALLOW, FLAG or DENY decision with a reason chain, and validate the system with seven scenarios and fifteen automated tests.

2. DOMAIN ANALYSIS

2.1 Domain Description and Requirements

The domain is access control in a university network. Whenever a person asks for a resource, the system looks at who the person is, which device is used, how the login went and how sensitive the resource is, and then decides what to do with the request. Table 2.1 lists the knowledge areas that the system has to represent.

Table 2.1: Knowledge areas of the CyberGuard domain

Area	Concepts	Examples in the system
Users	Student, Faculty, Administrator, IT Staff; role and failed login attempts	ali (student), sara (faculty), hamza (admin), zain (it_staff)
Devices	Trusted or unknown; managed or unmanaged	dev1 and dev2 are trusted; dev7 and dev9 are unknown
Resources	Public, internal, sensitive and critical; admin-only	course_page, student_db, grades_db, admin_panel
Security events	Login attempts, failed attempts, suspicious logins, role change attempts	login(ali, dev7), failed_attempts(ali, 4)
Threats	Brute force, unauthorized access, account compromise, privilege escalation	brute_force(ali), account_compromise(ali)

The functional requirements derived from the domain are:

1. **Brute force:** three or more failed login attempts classify the user as a brute-force attempt.
2. **Unknown device:** a login from a device that is not in the trusted list marks the device as unknown, and the login becomes suspicious.
3. **Risk:** a suspicious user who requests a sensitive or critical resource is a high-risk request.
4. **Unauthorized access:** a non-administrator who requests an admin-only resource is unauthorized.
5. **Threats:** brute force followed by a login from an unknown device is an account compromise; unauthorized access together with a role change attempt is a privilege escalation.
6. **Decision:** high-risk, unauthorized and compromised-account requests are denied; other suspicious requests are flagged; requests with nothing suspicious are allowed. If several decisions are derived, deny has priority over flag, and flag over allow.
7. **Explanation:** every decision must be traceable to the rules and facts that produced it.
8. **What-if analysis:** the user must be able to change a fact and run the reasoning again.

The non-functional requirements are explainability, deterministic traces (the same input always gives the same trace), the possibility to change rules without changing the engine, and execution with plain Python.

2.2 Entities, Relationships, and Challenges

The important entities are **User** (with the kinds Student, Faculty, Administrator and ITStaff), **Device** (trusted or unknown), **Resource** (public, internal, sensitive, critical and admin-only), **LoginEvent** and **Threat** (brute force, unauthorized access, account compromise and privilege escalation). They are connected by the following relationships: a user uses a device, a user requests a resource, a login event is by a user and from a device, and a threat targets a user. The kinds of user, device, resource and threat are related to their parent concept by is-a links (for example Student is-a User and CriticalResource is-a SensitiveResource). The complete structure is shown in Section 3.

Building a system for this domain raises the following challenges:

- **Derived knowledge.** Concepts such as brute_force, suspicious_login, high_risk and deny are never entered by a user. They have to be inferred from basic facts.
- **Combination of facts.** One fact alone is not enough. A login from an unknown device is only high risk when it also asks for a sensitive resource.

- **Absence of information.** An ALLOW decision needs the absence of suspicious signs. This requires negation as failure and the closed-world assumption, which is not pure Horn logic.
- **Competing conclusions.** A request can be flagged by one rule and denied by another. The rules therefore run in strata (levels) and the decision uses a priority.
- **Default knowledge.** An unlisted device is unknown by default, and a user without a recorded failure has zero failed attempts. Frames provide these defaults.
- **Expressive power against tractability.** FOL can state every rule but is only semi-decidable. Horn clauses are fast but cannot express disjunction or classical negation. The design must choose a workable balance (Section 8.2).

The project addresses the complex-problem attributes of the CCP as follows: **range of resources** (frames, semantic networks, FOL, DL, Horn clauses and chaining engines are used together), **level of interaction** (the trade-off between expressive power and tractability is analysed and resolved) and **innovation** (a rule-based knowledge structure for dynamic inference with automatic explanations).

A relational database is an excellent store for facts, but it is not designed to reason with them. Table 2.2 compares a traditional database with the knowledge-based approach used in CyberGuard.

Table 2.2: Traditional database compared with CyberGuard

Aspect	Traditional database	CyberGuard (KR and rules)
Derived knowledge	Returns only stored rows. high_risk or deny is not stored, so application code or views must compute it.	New facts are derived automatically by rules (for example R1 to R14).
Rules	Hard-coded in queries, triggers or application code. A policy change means changing and redeploying code.	Rules are data (Horn clauses). The engine is unchanged when a rule is added or edited.
Explanation	Gives an answer but no proof of why.	Prints a reason chain and a full forward or backward trace.
Goal-driven queries	Cannot start from a hypothesis such as “is deny justified?” and search for supporting facts.	Backward chaining starts from the hypothesis and proves it from facts.
Default values and inheritance	Column defaults only; no class hierarchy with overriding.	Frames give defaults, inheritance (Student is-a User) and demons.
Class-level reasoning	No built-in subsumption or disjointness checks.	DL axioms (Student and Administrator are disjoint) and a small DL reasoner.
Negation	Open to interpretation (NULL, NOT EXISTS).	Explicit negation as failure with stratified rules.

SQL views, stored procedures and triggers can compute some derived values, but the logic is scattered, hard to inspect and still gives no explanation. A database remains the right place to store large volumes of facts. In a complete deployment, the facts of CyberGuard would come from a database and the knowledge-based layer would reason over them.

3. KNOWLEDGE REPRESENTATION

Structural knowledge is represented in two complementary ways. **Frames** describe one concept in detail from the inside (slots, facets, defaults), while the **semantic network** shows how the concepts are connected to each other. Both are generated from the code (knowledge/frames.py and knowledge/semantic_net.py), so the diagrams always agree with the implementation.

3.1 Frames

A frame [2] is a class with **slots** (attributes), **facets** (rules about a slot: type, allowed values, range, default value, multiple values and an if-added demon) and an **is-a parent** from which it inherits. CyberGuard defines the frames User, Student, Faculty, Administrator, ITStaff, Device, Resource, LoginEvent and Threat. Student, Faculty, Administrator and ITStaff inherit all slots of User and only override the default of the slot role. The slot failed_attempts of User carries the demon warn_many_failures, which runs when a value is stored and reports that rule R1 (brute force) will match. Table 3.1 summarises the frames, and Figure 3.1 is the program output of Stage 1: it lists every frame with its slots and facets and shows which defaults are overridden by the child frames and which slots are inherited.

Table 3.1: Summary of the frames, slots, facets and default values

Frame	Is-a	Slots and facets	Default values
User	-	name (string); role (values: student, faculty, admin, it_staff); failed_attempts (integer, range 0..100, if-added demon warn_many_failures); uses (Device, multiple); requests (Resource, multiple)	role = student; failed_attempts = 0
Student	User	Inherits name, failed_attempts, uses, requests; overrides role	role = student
Faculty	User	Inherits all slots of User; overrides role	role = faculty
Administrator	User	Inherits all slots of User; overrides role	role = admin
ITStaff	User	Inherits all slots of User; overrides role	role = it_staff
Device	-	device_id (string); trust (values: trusted, unknown); management (values: managed, unmanaged)	trust = unknown; management = unmanaged
Resource	-	resource_id (string); sensitivity (values: public, internal, sensitive, critical); admin_only (boolean)	sensitivity = internal; admin_only = False
LoginEvent	-	user (User); device (Device); location (string); suspicious (boolean)	location = campus; suspicious = False
Threat	-	kind (values: brute_force, unauthorized, account_compromise, privilege_escalation); target (User); severity (values: low, medium, high)	severity = medium

```
================================================================================
S1: Brute force from an unknown device
================================================================================
Student ali fails the login 4 times from an unknown device and asks for the student database.

================================================================================
STAGE 1 of 6: FRAMES
================================================================================
What is this?
  A frame is a class that describes a concept. It has slots (attributes),
  facets (rules for a slot such as type, allowed values, range and a
  default value) and an is-a parent it inherits from. A script is the
  usual order of steps in a situation.

Frame: User
  name: type=string
  role: type=string, values=['student', 'faculty', 'admin', 'it_staff'], default=student
  failed_attempts: type=integer, default=0, range=0..100, if_added=warn_many_failures
  uses: type=Device, multiple=True
  requests: type=Resource, multiple=True
Frame: Student  (is-a User)
  role: default=student   <- overrides inherited default
  inherits: name, failed_attempts, uses, requests
Frame: Faculty  (is-a User)
  role: default=faculty   <- overrides inherited default
  inherits: name, failed_attempts, uses, requests
Frame: Administrator  (is-a User)
  role: default=admin   <- overrides inherited default
  inherits: name, failed_attempts, uses, requests
Frame: ITStaff  (is-a User)
  role: default=it_staff   <- overrides inherited default
  inherits: name, failed_attempts, uses, requests
Frame: Device
  device_id: type=string
  trust: type=string, values=['trusted', 'unknown'], default=unknown
  management: type=string, values=['managed', 'unmanaged'], default=unmanaged
Frame: Resource
  resource_id: type=string
  sensitivity: type=string, values=['public', 'internal', 'sensitive', 'critical'], default=internal
  admin_only: type=boolean, default=False
Frame: LoginEvent
  user: type=User
  device: type=Device
  location: type=string, default=campus
  suspicious: type=boolean, default=False
Frame: Threat
  kind: type=string, values=['brute_force', 'unauthorized', 'account_compromise', 'privilege_escalation']
  target: type=User
  severity: type=string, values=['low', 'medium', 'high'], default=medium

Frame architecture diagram saved to G:\CyberGuard\docs\frame_architecture.png

[Press Enter to see the script and the instances of this scenario]
```

Figure 3.1: Frame definitions printed by the program (Stage 1)

A script is the standard sequence of steps in a situation. The script “Login and Access Request” has five steps, and each step names the rules that apply to it. When a scenario runs, the frames are instantiated: slots that are not given take their default values, and the demon fires for failed_attempts = 4. Figure 3.2 shows the script and the frame instances created for scenario S1.

```
[Press Enter to see the script and the instances of this scenario]

--- Script ---
Script: Login and Access Request
  1. User enters credentials  ->  count failed_attempts, rule R1 (3 or more)
  2. System identifies the device  ->  trusted list lookup, rules R2 and R3
  3. User requests a resource  ->  requests(U, R) and sensitivity(R, S)
  4. System assesses risk and threats  ->  rules R4 to R9
  5. System decides deny, flag or allow  ->  rules R10 to R14

--- Instances built for this scenario (defaults are filled in automatically) ---
Instance of Student
  name = ali
  role = student
  failed_attempts = 4
  demon: failed_attempts = 4, rule R1 (brute force) will match
Instance of Device
  device_id = dev7
  trust = unknown
  management = unmanaged   (default)
Instance of LoginEvent
  user = ali
  device = dev7
  location = campus   (default)
  suspicious = False   (default)
Instance of Resource
  resource_id = student_db
  sensitivity = sensitive
  admin_only = False

[Press Enter for stage 2 (semantic network)]
```

Figure 3.2: The login script and the frame instances created for scenario S1

Figure 3.3 is the frame architecture diagram: blue arrows are is-a (inheritance) links and orange arrows are slots that point to another frame, similar to foreign keys in a database.

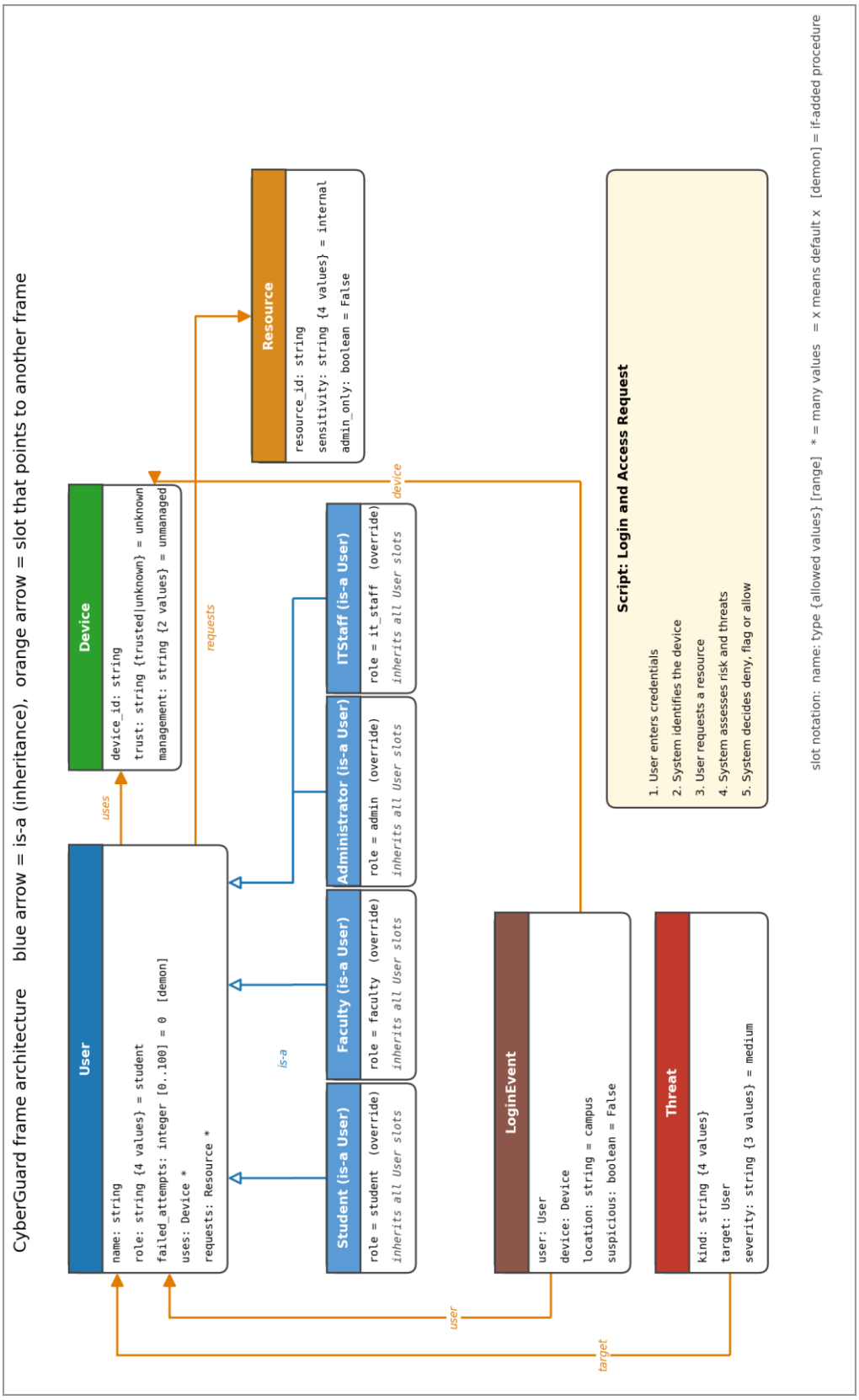


Figure 3.3: Frame architecture diagram of CyberGuard

3.2 Semantic Networks

The semantic network [2] connects concepts with labelled links: **is-a** (a kind of), **has-a** (owns a property) and **part-of** (belongs to a bigger whole), plus domain links such as uses, requests, by, from and targets. The is-a links are taken from the Description Logic subsumption axioms, so the network and the logic never disagree. In Figure 3.5 blue boxes are concepts, grey boxes are properties and green boxes are the

individuals of scenario S1 (ali, dev7 and student_db). Purple dashed links are instance-of links. The individuals change with the scenario, while the schema stays the same. Figure 3.4 is the textual form of the same network, printed by Stage 2.

```
================================================================================
STAGE 2 of 6: SEMANTIC NETWORK
================================================================================
What is this?
  A semantic network is a graph of concepts joined by labelled links: is-a
  (a kind of), has-a (owns a property) and part-of (belongs to a bigger
  whole). Frames describe one concept in detail, the network shows how the
  concepts connect.

is-a (a kind of):
  Student --is-a--> User
  Faculty --is-a--> User
  Administrator --is-a--> User
  ITStaff --is-a--> User
  TrustedDevice --is-a--> Device
  UnknownDevice --is-a--> Device
  SensitiveResource --is-a--> Resource
  CriticalResource --is-a--> SensitiveResource
  AdminOnlyResource --is-a--> CriticalResource
  BruteForce --is-a--> Threat
  AccountCompromise --is-a--> Threat
  PrivilegeEscalation --is-a--> Threat
  SuspiciousLogin --is-a--> LoginEvent

has-a (owns a property):
  User --has-a--> Role
  User --has-a--> FailedAttempts
  Device --has-a--> TrustLevel
  Resource --has-a--> Sensitivity
  LoginEvent --has-a--> Location
  Threat --has-a--> Severity

part-of (belongs to a bigger whole):
  Device --part-of--> CampusNetwork
  Resource --part-of--> CampusNetwork
  LoginEvent --part-of--> AccessRequest
  AccessRequest --part-of--> CampusNetwork

other links:
  LoginEvent --by--> User
  LoginEvent --from--> Device
  User --requests--> Resource
  User --uses--> Device
  Threat --targets--> User

individuals of this scenario:
  ali --instance-of--> Student
  ali --uses--> dev7
  dev7 --instance-of--> UnknownDevice
  ali --requests--> student_db
  student_db --instance-of--> SensitiveResource

Diagram saved to G:\CyberGuard\docs\semantic_network.png

[Press Enter for stage 3 (FOL and Description Logic)]
```

Figure 3.4: Relations of the semantic network and the individuals of S1 (Stage 2)

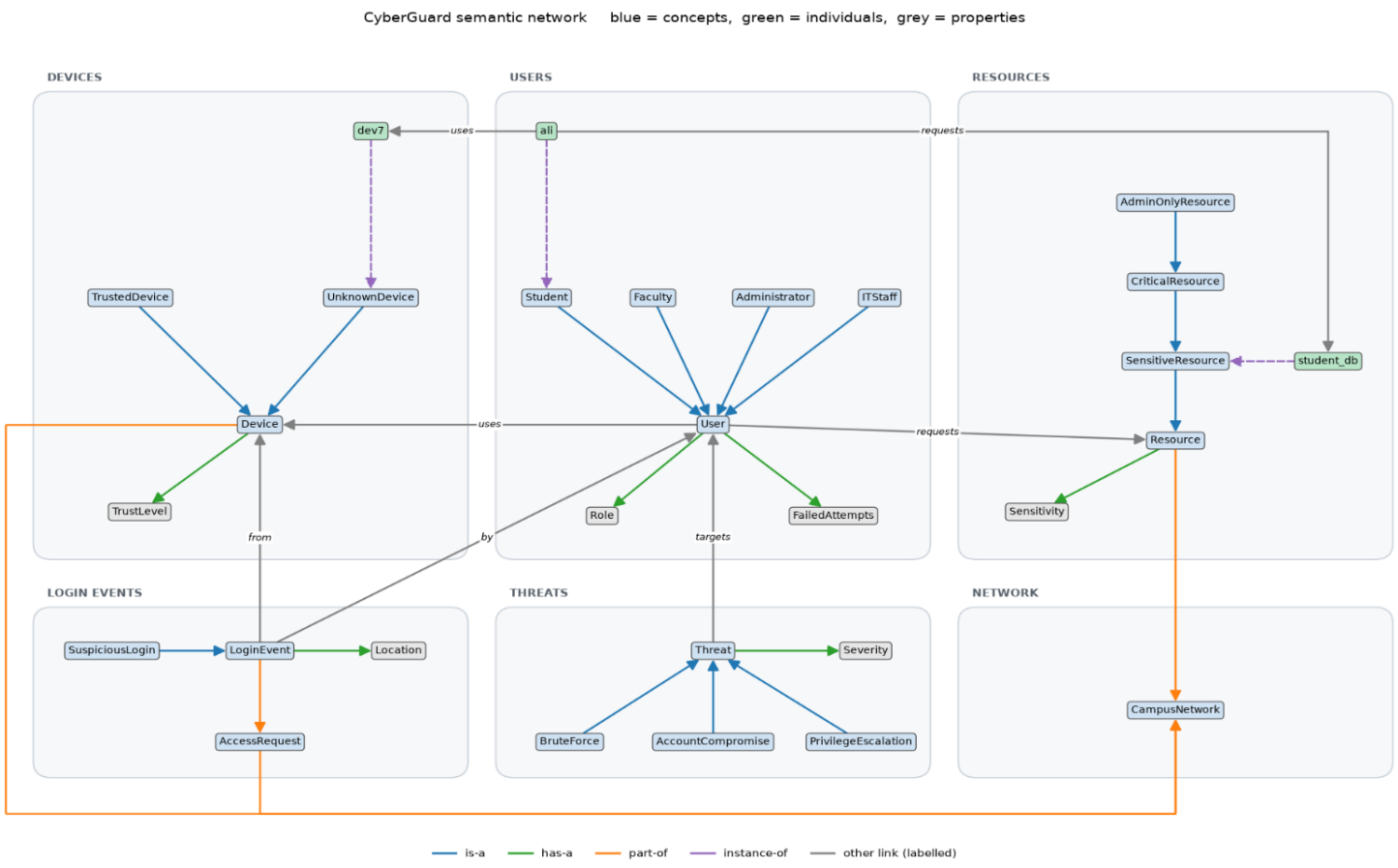


Figure 3.5: Semantic network of CyberGuard with the individuals of scenario S1

Table 3.2 compares the two structures. Neither of them derives new knowledge on its own. The value suspicious = False in the LoginEvent frame is only an initial state; the fact suspicious_login(ali) is derived later by the rules of Sections 4 and 5.

Table 3.2: Frame and semantic network

	Frame	Semantic network
View	One concept seen from the inside	Concepts and how they connect
Content of a node	Slots, facets, default values, demons	Only the name of the concept
Links	is-a parent and slots that point to other frames	is-a, has-a, part-of and domain links
Individuals	Instances built from the frame (ali = Student)	instance-of links (ali instance-of Student)
Database analogy	Table definition with constraints and defaults	Entity-relationship diagram

4. FORMAL LOGIC REPRESENTATION

4.1 First-Order Logic (FOL)

First-Order Logic [1] states the policy with predicates, constants, variables and quantifiers. Table 4.1 defines the predicates and the constants used in the domain. The variable **u** stands for a user, **d** for a device, **r** for a resource and **n** for a number.

Table 4.1: Predicates and constants of the CyberGuard domain

Predicate	Meaning
Role(u, x)	User u has the role x
FailedAttempts(u, n)	User u has n failed login attempts
Login(u, d)	User u logs in from device d
Trusted(d)	Device d is in the trusted list
Requests(u, r)	User u requests resource r
Sensitivity(r, s)	Resource r has the sensitivity level s
AdminOnly(r)	Resource r is reserved for administrators
RoleChangeAttempt(u)	User u tries to change a role
BruteForce(u), UnknownDevice(d), SuspiciousLogin(u)	Derived: brute-force user, unknown device, suspicious login
HighRisk(u, r), Unauthorized(u, r)	Derived: high-risk request, unauthorized request
AccountCompromise(u), PrivilegeEscalation(u)	Derived threats
Decision(u, r, x)	The decision for the request of u for r is x (allow, flag or deny)

Table 4.1 (continued): Constants of the CyberGuard domain

Type	Constants
Users	ali, sara, hamza, zain
Devices	dev1, dev2 (trusted); dev7, dev9 (not in the trusted list)
Resources	course_page, student_db, grades_db, admin_panel
Roles	student, faculty, admin, it_staff
Sensitivity levels	public, internal, sensitive, critical
Decisions	allow, flag, deny

The static facts of the domain are written in FOL as Role(ali, student), Role(sara, faculty), Role(hamza, admin), Role(zain, it_staff), Trusted(dev1), Trusted(dev2), Sensitivity(course_page, public), Sensitivity(student_db, sensitive), Sensitivity(grades_db, critical), Sensitivity(admin_panel, critical) and AdminOnly(admin_panel). The scenario S1 adds Login(ali, dev7), FailedAttempts(ali, 4) and Requests(ali, student_db).

A formula is read as “for all u and r, if the conditions on the left hold, then the conclusion on the right holds”. Symbols used: ∀ (for all), ∃ (there exists), ∧ (and), ∨ (or), ¬ (not), → (implies). Table 4.2 gives one formula for every rule of the knowledge base.

Table 4.2: FOL formulas of the rule base

Rule	FOL formula	Meaning
R1	$\forall u\ \forall n\ (\text{FailedAttempts}(u,n) \land n \geq 3 \rightarrow \text{BruteForce}(u))$	Three or more failed attempts means brute force
R2	$\forall u\ \forall d\ (\text{Login}(u,d) \land \neg\text{Trusted}(d) \rightarrow \text{UnknownDevice}(d))$	A login from a device that is not trusted marks it unknown
R3	$\forall u\ \forall d\ (\text{Login}(u,d) \land \text{UnknownDevice}(d) \rightarrow \text{SuspiciousLogin}(u))$	Login from an unknown device is suspicious
R4	$\forall u\ (\text{BruteForce}(u) \rightarrow \text{SuspiciousLogin}(u))$	A brute-force user is suspicious
R5	$\forall u\ \forall r\ (\text{SuspiciousLogin}(u) \land \text{Requests}(u,r) \land \text{Sensitivity}(r,\text{sensitive}) \rightarrow \text{HighRisk}(u,r))$	Suspicious user asks for a sensitive resource
R6	$\forall u\ \forall r\ (\text{SuspiciousLogin}(u) \land \text{Requests}(u,r) \land \text{Sensitivity}(r,\text{critical}) \rightarrow \text{HighRisk}(u,r))$	Suspicious user asks for a critical resource
R7	$\forall u\ \forall r\ (\text{Requests}(u,r) \land \text{AdminOnly}(r) \land \neg\text{Role}(u,\text{admin}) \rightarrow \text{Unauthorized}(u,r))$	A non-admin asks for an admin-only resource
R8	$\forall u\ \forall d\ (\text{BruteForce}(u) \land \text{Login}(u,d) \land \text{UnknownDevice}(d) \rightarrow \text{AccountCompromise}(u))$	Brute force followed by a login from an unknown device
R9	$\forall u\ \forall r\ (\text{Unauthorized}(u,r) \land \text{RoleChangeAttempt}(u) \rightarrow \text{PrivilegeEscalation}(u))$	Unauthorized access with a role change attempt
R10	$\forall u\ \forall r\ (\text{HighRisk}(u,r) \rightarrow \text{Decision}(u,r,\text{deny}))$	Deny a high-risk request
R11	$\forall u\ \forall r\ (\text{Unauthorized}(u,r) \rightarrow \text{Decision}(u,r,\text{deny}))$	Deny an unauthorized request
R12	$\forall u\ \forall r\ (\text{AccountCompromise}(u) \land \text{Requests}(u,r) \rightarrow \text{Decision}(u,r,\text{deny}))$	Deny every request of a compromised account
R13	$\forall u\ \forall r\ (\text{SuspiciousLogin}(u) \land \text{Requests}(u,r) \land \neg\text{Decision}(u,r,\text{deny}) \rightarrow \text{Decision}(u,r,\text{flag}))$	Suspicious but not denied: flag the request
R14	$\forall u\ \forall r\ (\text{Requests}(u,r) \land \neg\text{SuspiciousLogin}(u) \land \neg\text{Unauthorized}(u,r) \rightarrow \text{Decision}(u,r,\text{allow}))$	Nothing suspicious and authorized: allow

The formulas R2, R7, R13 and R14 contain ¬ in the body. Classical Horn clauses do not allow negation, so the engine reads ¬ as **negation as failure** (a condition that cannot be proved is treated as false), applied in strata. This is explained in Section 5. Table 4.3 lists the formulas that are outside the Horn fragment and how CyberGuard handles them.

Table 4.3: Formulas outside the Horn fragment and how CyberGuard handles them

Formula	Why it is not Horn	How it is handled
$\forall u\ (\text{User}(u) \rightarrow \text{Student}(u) \lor \text{Faculty}(u) \lor \text{Administrator}(u) \lor \text{ITStaff}(u))$	The head is a disjunction (more than one positive literal)	Every user gets exactly one role fact, so no disjunction is needed
$\exists u\ \exists r\ \text{Unauthorized}(u,r)$	As a knowledge base axiom it needs Skolem constants, so it is not used as a general rule	As a query it works with backward chaining, which treats u and r as variables and searches for values
$\forall u\ (\text{Student}(u) \rightarrow \neg\text{Administrator}(u))$	A negative conclusion is not allowed in a Horn head	Written as a DL disjointness axiom and checked by the DL reasoner

```
[Press Enter for stage 3 (FOL and Description Logic)]

================================================================================
STAGE 3 of 6: FIRST-ORDER LOGIC AND DESCRIPTION LOGIC
================================================================================
What is this?
  First-Order Logic states every rule with variables and quantifiers (for
  all u, for all r). Description Logic describes the classes: A is
  subsumed by B means every A is a B, and disjoint means nothing can be
  both. The Horn rules in the next stage are these formulas written in a
  form the engine can run.

First-Order Logic formulas (one for each Horn rule):
  R1  ∀u ∀n (FailedAttempts(u,n) ∧ n ≥ 3 → BruteForce(u))
      three or more failed attempts means brute force
  R2  ∀u ∀d (Login(u,d) ∧ ¬Trusted(d) → UnknownDevice(d))
      a login from a device that is not trusted marks it unknown
  R3  ∀u ∀d (Login(u,d) ∧ UnknownDevice(d) → SuspiciousLogin(u))
      login from an unknown device is suspicious
  R4  ∀u (BruteForce(u) → SuspiciousLogin(u))
      a brute force user is suspicious
  R5  ∀u ∀r (SuspiciousLogin(u) ∧ Requests(u,r) ∧ Sensitivity(r,sensitive) → HighRisk(u,r))
      suspicious user asking for a sensitive resource
  R6  ∀u ∀r (SuspiciousLogin(u) ∧ Requests(u,r) ∧ Sensitivity(r,critical) → HighRisk(u,r))
      suspicious user asking for a critical resource
  R7  ∀u ∀r (Requests(u,r) ∧ AdminOnly(r) ∧ ¬Role(u,admin) → Unauthorized(u,r))
      a non-admin asking for an admin-only resource
  R8  ∀u ∀d (BruteForce(u) ∧ Login(u,d) ∧ UnknownDevice(d) → AccountCompromise(u))
      brute force followed by a login from an unknown device
  R9  ∀u ∀r (Unauthorized(u,r) ∧ RoleChangeAttempt(u) → PrivilegeEscalation(u))
      unauthorized access together with a role change attempt
  R10 ∀u ∀r (HighRisk(u,r) → Decision(u,r,deny))
      deny a high risk request
  R11 ∀u ∀r (Unauthorized(u,r) → Decision(u,r,deny))
      deny an unauthorized request
  R12 ∀u ∀r (AccountCompromise(u) ∧ Requests(u,r) → Decision(u,r,deny))
      deny every request of a compromised account
  R13 ∀u ∀r (SuspiciousLogin(u) ∧ Requests(u,r) ∧ ¬Decision(u,r,deny) → Decision(u,r,flag))
      suspicious but not denied: flag the event
  R14 ∀u ∀r (Requests(u,r) ∧ ¬SuspiciousLogin(u) ∧ ¬Unauthorized(u,r) → Decision(u,r,allow))
      nothing suspicious and authorized: allow

Note: R2, R7, R13 and R14 have ¬ in the body. Classical Horn clauses allow no negation,
so the engine reads ¬ as negation as failure (stratified), see stage 4.

Formulas that are NOT Horn:
  ∀u (User(u) → Student(u) ∨ Faculty(u) ∨ Administrator(u) ∨ ITStaff(u))
      why: the head is a disjunction (more than one positive literal)
      handled: every user gets exactly one role fact, so no disjunction is needed
  ∃u ∃r Unauthorized(u,r)
      why: as a knowledge base axiom it needs Skolem constants, an existential cannot be a Horn clause
      handled: as a query it is fine: backward chaining treats u and r as variables and searches for values
  ∀u (Student(u) → ¬Administrator(u))
      why: a negative conclusion is not allowed in a Horn head
      handled: written as a DL disjointness axiom and checked by the DL reasoner

[Press Enter to see the Description Logic axioms]
```

Figure 4.1: First-order logic formulas printed by the program (Stage 3)

4.2 Description Logic (DL)

Description Logic [2] describes the classes of the domain. The **TBox** holds the concept axioms. **Subsumption** $A \sqsubseteq B$ means that every A is a B. **Disjointness** $A \sqcap B \sqsubseteq \bot$ means that nothing can be both A and B. The **ABox** holds the memberships of individuals, such as Student(ali). The concepts are the classes of the domain (User, Student, Device, Resource, Threat and so on) and the roles are uses, requests, hasFailedAttempts, by, from and targets.

Table 4.4: Subsumption axioms (TBox)

Axiom (1)	Axiom (2)
Student ⊑ User	Faculty ⊑ User
Administrator ⊑ User	ITStaff ⊑ User
TrustedDevice ⊑ Device	UnknownDevice ⊑ Device
SensitiveResource ⊑ Resource	CriticalResource ⊑ SensitiveResource
AdminOnlyResource ⊑ CriticalResource	BruteForce ⊑ Threat
AccountCompromise ⊑ Threat	PrivilegeEscalation ⊑ Threat
SuspiciousLogin ⊑ LoginEvent	

Table 4.5: Disjointness axioms (TBox)

Axiom (1)	Axiom (2)
Student ⊓ Faculty ⊑ ⊥	Student ⊓ Administrator ⊑ ⊥
Student ⊓ ITStaff ⊑ ⊥	Faculty ⊓ Administrator ⊑ ⊥
Faculty ⊓ ITStaff ⊑ ⊥	Administrator ⊓ ITStaff ⊑ ⊥
TrustedDevice ⊓ UnknownDevice ⊑ ⊥	User ⊓ Device ⊑ ⊥
User ⊓ Resource ⊑ ⊥	Device ⊓ Resource ⊑ ⊥

Table 4.6: Defined classes and role axioms

Axiom	Reading
BruteForceUser ≡ User ⊓ ∃hasFailedAttempts.AtLeastThree	A user with at least three failed attempts
SuspiciousUser ≡ User ⊓ (∃uses.UnknownDevice ⊔ BruteForceUser)	A user of an unknown device, or a brute-force user
UnauthorizedUser ≡ User ⊓ ¬Administrator ⊓ ∃requests.AdminOnlyResource	A non-admin who requests an admin-only resource
∃uses.⊤ ⊑ User	Domain of uses is User
⊤ ⊑ ∀uses.Device	Range of uses is Device
⊤ ⊑ ∀requests.Resource	Range of requests is Resource

Note on naming. In the TBox, the concepts BruteForce, AccountCompromise, PrivilegeEscalation and SuspiciousLogin are classes of threat and login-event individuals. In the FOL and Horn rules of Sections 4.1 and 5, the predicates brute_force(u), account_compromise(u), privilege_escalation(u) and suspicious_login(u) are applied directly to the user, which keeps the rules short. The inference of Sections 5 and 6 uses only the user-level predicates, and Description Logic is used only for class-level checks (subsumption, disjointness and ABox consistency).

The memberships (ABox) of scenario S1 are Student(ali), UnknownDevice(dev7), SensitiveResource(student_db), uses(ali, dev7) and requests(ali, student_db). The reasoner adds the inherited classes (ali is also a User, dev7 a Device, student_db a Resource) and checks that no individual belongs to two disjoint classes. The consistency check passes.

The small DL reasoner answers subsumption and disjointness questions. For example, CriticalResource ⊑ Resource is **true** (the chain Critical ⊑ Sensitive ⊑ Resource), Resource ⊑ CriticalResource is **false**,

Student and Administrator are **disjoint**, while Student and User are **not** disjoint. Figure 4.2 shows the axioms, the reasoning answers and the ABox check printed by the program.

```
[Press Enter to see the Description Logic axioms]
Description Logic TBox:
  Subsumption (A ⊑ B, every A is a B):
    Student ⊑ User
    Faculty ⊑ User
    Administrator ⊑ User
    ITStaff ⊑ User
    TrustedDevice ⊑ Device
    UnknownDevice ⊑ Device
    SensitiveResource ⊑ Resource
    CriticalResource ⊑ SensitiveResource
    AdminOnlyResource ⊑ CriticalResource
    BruteForce ⊑ Threat
    AccountCompromise ⊑ Threat
    PrivilegeEscalation ⊑ Threat
    SuspiciousLogin ⊑ LoginEvent
  Disjointness (A ⊓ B ⊑ ⊥, nothing is both):
    Student ⊓ Faculty ⊑ ⊥
    Student ⊓ Administrator ⊑ ⊥
    Student ⊓ ITStaff ⊑ ⊥
    Faculty ⊓ Administrator ⊑ ⊥
    Faculty ⊓ ITStaff ⊑ ⊥
    Administrator ⊓ ITStaff ⊑ ⊥
    TrustedDevice ⊓ UnknownDevice ⊑ ⊥
    User ⊓ Device ⊑ ⊥
    User ⊓ Resource ⊑ ⊥
    Device ⊓ Resource ⊑ ⊥
  Definitions and properties:
    BruteForceUser ≡ User ⊓ ∃hasFailedAttempts.AtLeastThree
    SuspiciousUser ≡ User ⊓ (∃uses.UnknownDevice ⊔ BruteForceUser)
    UnauthorizedUser ≡ User ⊓ ¬Administrator ⊓ ∃requests.AdminOnlyResource
    ∃uses.⊤ ⊑ User          (domain of uses)
    ⊤ ⊑ ∀uses.Device        (range of uses)
    ⊤ ⊑ ∀requests.Resource (range of requests)

DL reasoning (answered by the small reasoner in this file):
  Is CriticalResource ⊑ Resource?  YES
  Is AdminOnlyResource ⊑ SensitiveResource?  YES
  Is Resource ⊑ CriticalResource?  NO
  Are Student and Administrator disjoint?  YES
  Are TrustedDevice and UnknownDevice disjoint?  YES
  Are Student and User disjoint?  NO

ABox (individuals of this scenario):
  ali : Student, User
  dev7 : UnknownDevice, Device
  student_db : SensitiveResource, Resource
  Consistency check: OK, no individual is in two disjoint classes

[Press Enter to see the limitations]
```

Figure 4.2: Description Logic axioms, reasoning answers and ABox check (Stage 3)

5. HORN CLAUSE KNOWLEDGE BASE

A Horn clause [1] has at most one positive conclusion: head :- condition1, condition2. It is read as “if condition1 and condition2 hold, then head holds”. The comma means “and”, and a variable starts with ?. The FOL formulas of Section 4.1 become Horn clauses in three steps: (1) drop the universal quantifiers, because every variable is universally quantified; (2) write the implication with the conclusion first; (3) read ¬ in the body as not, which is negation as failure. For example, R5 in clause form is ¬SuspiciousLogin(u) ∨ ¬Requests(u,r) ∨ ¬Sensitivity(r,sensitive) ∨ HighRisk(u,r). It has exactly one positive literal, so it is a Horn clause.

Rules are placed in three **strata** (levels). A rule that uses not runs only after every fact it refers to has been completely derived. Without this, the result would depend on the order of the rules. Stratum 0 classifies and assesses (R1 to R9), stratum 1 derives the deny decisions (R10 to R12), and stratum 2 derives flag and allow (R13 and R14), which use not on facts of the earlier strata. Two rules with the same head, such as R3 and R4, mean “or”. The predicate decision appears in strata 1 and 2, and R13 uses not decision(?U, ?R, deny). At predicate level this is a negative self-dependency, but the rule base is locally stratified: R13 negates only the deny value, and all deny decisions are completely derived in stratum 1 before the flag and allow decisions are derived in stratum 2.

Table 5.1: Horn clause rule base (14 rules in 3 strata)

Rule	Horn clause	Stratum
R1	brute_force(?U) :- failed_attempts(?U, ?N), ?N >= 3	0
R2	unknown_device(?D) :- login(?U, ?D), not trusted(?D)	0
R3	suspicious_login(?U) :- login(?U, ?D), unknown_device(?D)	0
R4	suspicious_login(?U) :- brute_force(?U)	0
R5	high_risk(?U, ?R) :- suspicious_login(?U), requests(?U, ?R), sensitivity(?R, sensitive)	0
R6	high_risk(?U, ?R) :- suspicious_login(?U), requests(?U, ?R), sensitivity(?R, critical)	0
R7	unauthorized(?U, ?R) :- requests(?U, ?R), admin_only(?R), not role(?U, admin)	0
R8	account_compromise(?U) :- brute_force(?U), login(?U, ?D), unknown_device(?D)	0
R9	privilege_escalation(?U) :- unauthorized(?U, ?R), role_change_attempt(?U)	0
R10	decision(?U, ?R, deny) :- high_risk(?U, ?R)	1
R11	decision(?U, ?R, deny) :- unauthorized(?U, ?R)	1
R12	decision(?U, ?R, deny) :- account_compromise(?U), requests(?U, ?R)	1
R13	decision(?U, ?R, flag) :- suspicious_login(?U), requests(?U, ?R), not decision(?U, ?R, deny)	2
R14	decision(?U, ?R, allow) :- requests(?U, ?R), not suspicious_login(?U), not unauthorized(?U, ?R)	2

The static facts of the knowledge base are given in Table 5.2. The scenario S1 adds three input facts: login(ali, dev7), failed_attempts(ali, 4) and requests(ali, student_db). Figure 5.1 shows the knowledge base as printed by Stage 4.

Table 5.2: Static facts of the knowledge base

Group	Facts
Roles	role(ali, student), role(sara, faculty), role(hamza, admin), role(zain, it_staff)
Resources	sensitivity(course_page, public), sensitivity(student_db, sensitive), sensitivity(grades_db, critical), sensitivity(admin_panel, critical), admin_only(admin_panel)

Group	Facts
Trusted devices	trusted(dev1), trusted(dev2). Any other device is unknown (rule R2).

```
======================================================================
STAGE 4 of 6: HORN CLAUSE KNOWLEDGE BASE
======================================================================
What is this?
  A Horn clause has at most one positive conclusion: head :- condition1,
  condition2. That restriction makes reasoning fast. Rules with 'not' use
  negation as failure: something that cannot be proved is treated as
  false. Stratum is the level at which a rule runs, so 'not' is checked
  only after the facts it refers to are complete.


--- Static facts ---
Roles:
  role(ali, student)
  role(sara, faculty)
  role(hamza, admin)
  role(zain, it_staff)
Resources:
  sensitivity(course_page, public)
  sensitivity(student_db, sensitive)
  sensitivity(grades_db, critical)
  sensitivity(admin_panel, critical)
  admin_only(admin_panel)
Trusted devices:
  trusted(dev1)
  trusted(dev2)

--- Rules ---
Stratum 0: classification, risk and threats
  R1_brute_force: brute_force(?U) :- failed_attempts(?U, ?N), ?N >= 3   [Horn]
  R2_unknown_device: unknown_device(?D) :- login(?U, ?D), not trusted(?D)   [Horn + not]
  R3_suspicious_unknown_device: suspicious_login(?U) :- login(?U, ?D), unknown_device(?D)   [Horn]
  R4_suspicious_brute_force: suspicious_login(?U) :- brute_force(?U)   [Horn]
  R5_high_risk_sensitive: high_risk(?U, ?R) :- suspicious_login(?U), requests(?U, ?R), sensitivity(?R, sensitive)   [Horn]
  R6_high_risk_critical: high_risk(?U, ?R) :- suspicious_login(?U), requests(?U, ?R), sensitivity(?R, critical)   [Horn]
  R7_unauthorized: unauthorized(?U, ?R) :- requests(?U, ?R), admin_only(?R), not role(?U, admin)   [Horn + not]
  R8_account_compromise: account_compromise(?U) :- brute_force(?U), login(?U, ?D), unknown_device(?D)   [Horn]
  R9_privilege_escalation: privilege_escalation(?U) :- unauthorized(?U, ?R), role_change_attempt(?U)   [Horn]

Stratum 1: deny decisions
  R10_deny_high_risk: decision(?U, ?R, deny) :- high_risk(?U, ?R)   [Horn]
  R11_deny_unauthorized: decision(?U, ?R, deny) :- unauthorized(?U, ?R)   [Horn]
  R12_deny_compromised: decision(?U, ?R, deny) :- account_compromise(?U), requests(?U, ?R)   [Horn]

Stratum 2: flag and allow decisions
  R13_flag: decision(?U, ?R, flag) :- suspicious_login(?U), requests(?U, ?R), not decision(?U, ?R, deny)   [Horn + not]
  R14_allow: decision(?U, ?R, allow) :- requests(?U, ?R), not suspicious_login(?U), not unauthorized(?U, ?R)   [Horn + not]


--- Input facts of this scenario ---
  login(ali, dev7)
  failed_attempts(ali, 4)
  requests(ali, student_db)
```

Figure 5.1: Horn clause knowledge base printed by the program (Stage 4)

6. INFERENCE AND REASONING

6.1 Forward Chaining

Forward chaining [1] starts from the known facts. It fires every rule whose conditions are true, adds the new fact, and repeats until no new fact can be derived (a fixpoint). It runs stratum by stratum. Scenario S1: student ali fails the login four times from the unknown device dev7 and asks for the sensitive student database. The knowledge base starts with 14 facts (11 static and 3 from the scenario). Table 6.1 shows the rules activated and each newly derived fact.

Table 6.1: Forward chaining trace of scenario S1

Step	Rule fired	Conditions that hold	New fact derived
1	R1_brute_force	failed_attempts(ali, 4) and 4 >= 3	brute_force(ali)
2	R2_unknown_device	login(ali, dev7) and not trusted(dev7)	unknown_device(dev7)
3	R3_suspicious_unknown_device	login(ali, dev7) and unknown_device(dev7)	suspicious_login(ali)
4	R5_high_risk_sensitive	suspicious_login(ali), requests(ali, student_db), sensitivity(student_db, sensitive)	high_risk(ali, student_db)
5	R8_account_compromise	brute_force(ali), login(ali, dev7), unknown_device(dev7)	account_compromise(ali)
6	R10_deny_high_risk	high_risk(ali, student_db)	decision(ali, student_db, deny)

No new fact can be derived after step 6, so the process stops with six new facts and the final conclusion **deny**. R4 does not appear in the trace because suspicious_login(ali) had already been derived by R3. R12 does not appear because decision(ali, student_db, deny) already existed after R10. Neither rule produced a new fact.

```
======================================================================
STAGE 5 of 6: FORWARD CHAINING
======================================================================
What is this?
  Forward chaining is data driven. It starts from the known facts, fires
  every rule whose conditions are true, adds the new fact, and repeats
  until nothing new can be derived.

Known facts at the start: 14 (11 static + 3 from the scenario)

[FORWARD  1] R1_brute_force: failed_attempts(ali, 4) + 4 >= 3  =>  brute_force(ali)
[FORWARD  2] R2_unknown_device: login(ali, dev7) + not trusted(dev7)  =>  unknown_device(dev7)
[FORWARD  3] R3_suspicious_unknown_device: login(ali, dev7) + unknown_device(dev7)  =>  suspicious_login(ali)
[FORWARD  4] R5_high_risk_sensitive: suspicious_login(ali) + requests(ali, student_db) + sensitivity(student_db, sensitive)  =>  high_risk(ali, student_db)
[FORWARD  5] R8_account_compromise: brute_force(ali) + login(ali, dev7) + unknown_device(dev7)  =>  account_compromise(ali)
[FORWARD  6] R10_deny_high_risk: high_risk(ali, student_db)  =>  decision(ali, student_db, deny)

No new fact can be derived. 6 new facts in total.
Decisions derived for ali / student_db: deny

[Press Enter for stage 6 (backward chaining)]
```

Figure 6.1: Forward chaining trace of scenario S1 (Stage 5)

Scenario S3 is a normal case: the administrator hamza logs in from the trusted device dev1 and opens the student database. It shows how the system reaches an ALLOW decision, which depends on negation as failure. Only one rule fires (Table 6.2).

Table 6.2: Forward chaining trace of scenario S3

Step	Rule fired	Conditions that hold	New fact derived
1	R14_allow	requests(hamza, student_db), not suspicious_login(hamza), not unauthorized(hamza, student_db)	decision(hamza, student_db, allow)

```
================================================================
STAGE 5 of 6: FORWARD CHAINING
================================================================
What is this?
  Forward chaining is data driven. It starts from the known facts, fires
  every rule whose conditions are true, adds the new fact, and repeats
  until nothing new can be derived.

Known facts at the start: 14 (11 static + 3 from the scenario)

[FORWARD  1] R14_allow: requests(hamza, student_db) + not suspicious_login(hamza) + not unauthorized(hamza, student_db)  =>  decision(hamza, student_db, allow)

No new fact can be derived. 1 new facts in total.
Decisions derived for hamza / student_db: allow

[Press Enter for stage 6 (backward chaining)]
```

Figure 6.2: Forward chaining trace of scenario S3 (Stage 5)

6.2 Backward Chaining

Backward chaining starts from a hypothesis. For the goal decision(ali, student_db, deny) it finds a rule whose head matches the goal, makes each condition of that rule a sub-goal, and repeats until every branch ends in a known fact. A not condition succeeds when the negated goal cannot be proved. The engine renames the variables of a rule at every use (for example ?U becomes ?U_1), so bindings from different uses of the same rule do not clash. Table 6.3 shows the proof tree.

Table 6.3: Proof tree of the goal decision(ali, student_db, deny)

Level	Goal	Proved by	Sub-goals and facts
1	decision(ali, student_db, deny)	Rule R10	high_risk(ali, student_db)
2	high_risk(ali, student_db)	Rule R5	suspicious_login(ali); requests(ali, student_db) [fact]; sensitivity(student_db, sensitive) [fact]
3	suspicious_login(ali)	Rule R3	login(ali, dev7) [fact]; unknown_device(dev7)
4	unknown_device(dev7)	Rule R2	login(ali, dev7) [fact]; not trusted(dev7)
5	not trusted(dev7)	Negation as failure	trusted(dev7) cannot be proved (FAILED), so the “not” holds

Every branch ends in a given fact, so the goal is **proved**. The engine stops at the first successful rule (R10), which is why R11 and R12 are not tried. The backward decision (deny) is equal to the forward decision, so the two methods agree. The reason chain at the end of Figure 6.3 is the explanation of the decision. It is built from the forward trace by following every derived fact back to the rule and the conditions that created it, until only given facts are left.

```
================================================================================
STAGE 6 of 6: BACKWARD CHAINING
================================================================================
What is this?
  Backward chaining is goal driven. It starts from a hypothesis, for
  example decision(ali, student_db, deny), finds a rule that could prove
  it, and then tries to prove each condition of that rule. It stops when
  every branch ends in a known fact.

Goal: decision(ali, student_db, deny)

GOAL decision(ali, student_db, deny)
  RULE R10_deny_high_risk
    GOAL high_risk(ali, student_db)
      RULE R5_high_risk_sensitive
        GOAL suspicious_login(ali)
          RULE R3_suspicious_unknown_device
            GOAL login(ali, ?D)
              FACT login(ali, dev7)  (given)
            GOAL unknown_device(dev7)
              RULE R2_unknown_device
                GOAL login(?U, dev7)
                  FACT login(ali, dev7)  (given)
                NOT trusted(dev7) ?
                  GOAL trusted(dev7)
                    FAILED trusted(dev7)
                NOT trusted(dev7) -> True
              PROVED unknown_device(dev7) by R2_unknown_device
          PROVED suspicious_login(ali) by R3_suspicious_unknown_device
        GOAL requests(ali, student_db)
          FACT requests(ali, student_db)  (given)
        GOAL sensitivity(student_db, sensitive)
          FACT sensitivity(student_db, sensitive)  (given)
      PROVED high_risk(ali, student_db) by R5_high_risk_sensitive
  PROVED decision(ali, student_db, deny) by R10_deny_high_risk

PROVED: decision(ali, student_db, deny)

--- Final result ---
Forward chaining decision : DENY
Backward chaining decision: DENY
Both methods agree.

Why this decision (reason chain):
  decision(ali, student_db, deny)   <= R10_deny_high_risk
    high_risk(ali, student_db)   <= R5_high_risk_sensitive
      suspicious_login(ali)   <= R3_suspicious_unknown_device
        login(ali, dev7)   [given fact]
        unknown_device(dev7)   <= R2_unknown_device
          login(ali, dev7)   [given fact]
          not trusted(dev7)   [cannot be proved, so the 'not' holds]
      requests(ali, student_db)   [given fact]
      sensitivity(student_db, sensitive)   [given fact]

FINAL DECISION: DENY   (expected DENY: PASS)
```

Figure 6.3: Backward chaining proof tree, final result and reason chain of scenario S1 (Stage 6)

```
================================================================================
STAGE 6 of 6: BACKWARD CHAINING
================================================================================
What is this?
  Backward chaining is goal driven. It starts from a hypothesis, for
  example decision(ali, student_db, deny), finds a rule that could prove
  it, and then tries to prove each condition of that rule. It stops when
  every branch ends in a known fact.

Goal: decision(hamza, student_db, deny)  ->  cannot be proved, try the next decision

Goal: decision(hamza, student_db, flag)  ->  cannot be proved, try the next decision

Goal: decision(hamza, student_db, allow)

GOAL decision(hamza, student_db, allow)
  RULE R14_allow
    GOAL requests(hamza, student_db)
      FACT requests(hamza, student_db)  (given)
    NOT suspicious_login(hamza) ?
      GOAL suspicious_login(hamza)
        RULE R3_suspicious_unknown_device
          GOAL login(hamza, ?D)
            FACT login(hamza, dev1)  (given)
          GOAL unknown_device(dev1)
            RULE R2_unknown_device
              GOAL login(?U, dev1)
                FACT login(hamza, dev1)  (given)
              NOT trusted(dev1) ?
                GOAL trusted(dev1)
                  FACT trusted(dev1)  (given)
              NOT trusted(dev1) -> False
            FAILED unknown_device(dev1)
        RULE R4_suspicious_brute_force
          GOAL brute_force(hamza)
            RULE R1_brute_force
              GOAL failed_attempts(hamza, ?N)
                FACT failed_attempts(hamza, 0)  (given)
              CHECK 0 >= 3 -> False
            FAILED brute_force(hamza)
        FAILED suspicious_login(hamza)
    NOT suspicious_login(hamza) -> True
    NOT unauthorized(hamza, student_db) ?
      GOAL unauthorized(hamza, student_db)
        RULE R7_unauthorized
          GOAL requests(hamza, student_db)
            FACT requests(hamza, student_db)  (given)
          GOAL admin_only(student_db)
            FAILED admin_only(student_db)
        FAILED unauthorized(hamza, student_db)
    NOT unauthorized(hamza, student_db) -> True
  PROVED decision(hamza, student_db, allow) by R14_allow

PROVED: decision(hamza, student_db, allow)

--- Final result ---
Forward chaining decision : ALLOW
Backward chaining decision: ALLOW
Both methods agree.
```

Figure 6.4: Backward chaining trace of scenario S3 (Stage 6)

For scenario S3, the engine tries the decisions in priority order. The goals deny and flag **cannot be proved**, and the goal allow is proved by R14:

- **Deny** fails: there is no high_risk, unauthorized or account_compromise fact for hamza.
- **Flag** fails: suspicious_login(hamza) cannot be proved. R3 fails because dev1 is trusted, so unknown_device(dev1) fails. R4 fails because brute_force(hamza) cannot be proved: R1 fails, since failed_attempts(hamza, 0) does not satisfy 0 >= 3.
- **Allow** succeeds with R14: not suspicious_login(hamza) holds and not unauthorized(hamza, student_db) holds, because admin_only(student_db) fails.

```
Why this decision (reason chain):
  decision(hamza, student_db, allow)   <= R14_allow
    requests(hamza, student_db)   [given fact]
    not suspicious_login(hamza)   [cannot be proved, so the 'not' holds]
    not unauthorized(hamza, student_db)   [cannot be proved, so the 'not' holds]

FINAL DECISION: ALLOW   (expected ALLOW: PASS)

================================================================================
CONCLUSION
================================================================================
                                                    FINAL DECISION:  ALLOW
                                                    PASS  (expected ALLOW)


  Forward chaining  (data driven) : ALLOW
  Backward chaining (goal driven) : ALLOW
  Both methods agree.

  Rules that fired, in order : R14
  New facts derived          : 1   (none of them was stored, all were derived)
  Contrast                   : 'S1: Brute force from an unknown device' gives DENY
```

Scenarios S1 and S3 show that the system denies risky requests and allows normal ones. Both forward and backward chaining produce the same decisions.

7. IMPLEMENTATION

7.1 Tools and Technologies

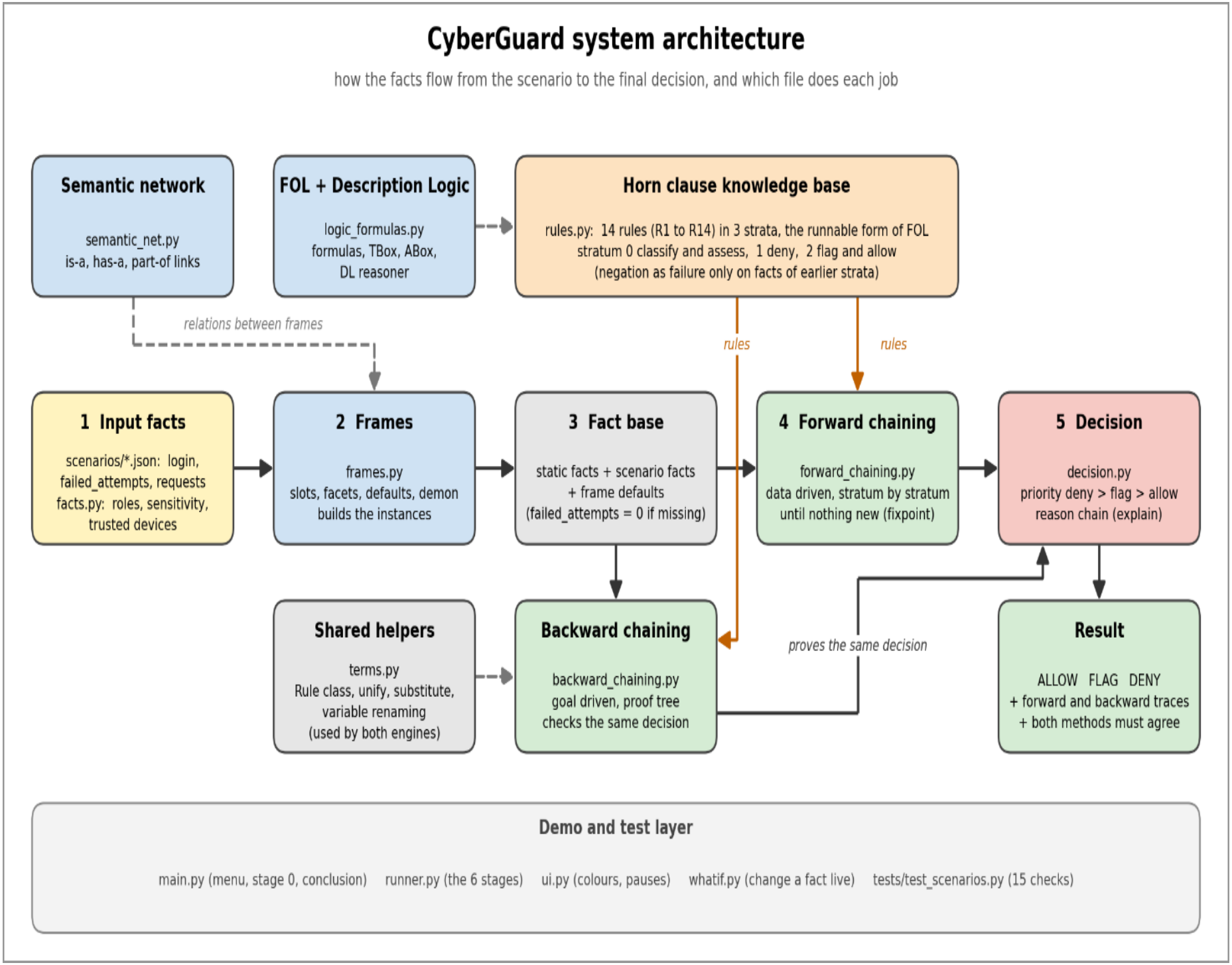
The system is implemented as a **Python rule engine** [3]. Facts are tuples, for example ("role", "ali", "student"), and a rule is an object with a head, a body, a stratum and a documentation string. The engine consists of unification (unify), forward chaining, backward chaining and a decision module. The program requires only Python 3 and the library matplotlib [4] (for the two diagrams). The inference is done by the team's own engine and not by an external reasoning library. Figure 7.1 shows how the facts flow through the system: the scenario and the static facts become frame instances, which are put into the fact base together with the default values. Forward chaining derives all consequences with the Horn rules, and backward chaining proves the same decision from the goal. Both engines share the helper functions of terms.py, and the decision module picks the result with the priority deny, flag, allow.

Table 7.1: Project structure

```
CyberGuard/
|-- main.py                 demo menu, stage 0 and conclusion
|-- requirements.txt        matplotlib
|-- make_architecture.py    draws the system architecture diagram
|-- demo/
|   |-- runner.py           the six demo stages
|   |-- ui.py               colours, headings and pauses
|   +-- whatif.py           change a fact and reason again
|-- knowledge/
|   |-- frames.py           frames, script, instances, frame diagram
|   |-- semantic_net.py     semantic network and its diagram
|   |-- logic_formulas.py   FOL formulas, DL axioms, DL reasoner
|   |-- facts.py            static facts
|   +-- rules.py            14 Horn rules in 3 strata
|-- engine/
|   |-- terms.py            Rule class, unification, helpers
|   |-- forward_chaining.py data-driven reasoning
|   |-- backward_chaining.py goal-driven reasoning
|   +-- decision.py         ALLOW / FLAG / DENY and the explanation
|-- scenarios/              seven scenario files (s1 to s7, JSON)
|-- tests/test_scenarios.py 15 automated checks
+-- docs/                   generated diagrams
```

Table 7.2: How to run the system

Command	Purpose
pip install matplotlib	Install the only required library
python main.py	Open the menu and choose a scenario
python main.py 1	Run scenario 1 through all stages (pauses after every stage)
python main.py 1 --auto	Run scenario 1 without pauses (for the report output)
python main.py all	Check all seven scenarios and print a summary table
python tests/test_scenarios.py	Run the 15 automated tests


CyberGuard system architecture
how the facts flow from the scenario to the final decision, and which file does each job
Semantic network
semantic_net.py
is-a, has-a, part-of links
FOL + Description Logic
logic_formulas.py
formulas, TBox, ABox,
DL reasoner
Horn clause knowledge base
rules.py: 14 rules (R1 to R14) in 3 strata, the runnable form of FOL
stratum 0 classify and assess, 1 deny, 2 flag and allow
(negation as failure only on facts of earlier strata)
relations between frames
rules
rules
1 Input facts
scenarios/*.json: login,
failed_attempts, requests
facts.py: roles, sensitivity,
trusted devices
2 Frames
frames.py
slots, facets, defaults, demon
builds the instances
3 Fact base
static facts + scenario facts
+ frame defaults
(failed_attempts = 0 if missing)
4 Forward chaining
forward_chaining.py
data driven, stratum by stratum
until nothing new (fixpoint)
5 Decision
decision.py
priority deny > flag > allow
reason chain (explain)
Shared helpers
terms.py
Rule class, unify, substitute,
variable renaming
(used by both engines)
Backward chaining
backward_chaining.py
goal driven, proof tree
checks the same decision
proves the same decision
Result
ALLOW FLAG DENY
+ forward and backward traces
+ both methods must agree
Demo and test layer
main.py (menu, stage 0, conclusion)
runner.py (the 6 stages)
ui.py (colours, pauses)
whatif.py (change a fact live)
tests/test_scenarios.py (15 checks)

7.2 Code / Execution

This section shows the execution of the program and the key parts of the rule engine. The complete source code is part of the submitted project folder. The program opens with a menu of the seven scenarios (Figure 7.2). Each scenario shows its expected decision in colour (red for DENY, yellow for FLAG and green for ALLOW). Before the six reasoning stages, Stage 0 states the domain, the problem, why a normal database is not enough, and who explains which stage (Figure 7.3). The output of the stages of scenario S1 is shown in Figures 3.1, 3.2, 3.4, 4.1, 4.2, 5.1, 6.1 and 6.3.



Figure 7.2: Main menu of CyberGuard

```
STAGE 0: PROBLEM AND MOTIVATION
================================================================================

--- Domain ---
  University cybersecurity access control. For every access request the
  system must decide ALLOW, FLAG or DENY.

--- Problem ---
  The decision depends on several facts together (failed attempts, device,
  resource sensitivity, role), not on one stored value.

--- Why a normal database is not enough ---
  - A database only returns stored data. high_risk or deny is never stored,
    it has to be derived by rules.
  - Rules are hard-coded in queries or application code, so changing a
    rule means changing code.
  - A database cannot explain WHY a decision was made. This system
    prints the full reason chain.

--- What we build ---
  Frames + Semantic Network (structure), First-Order Logic + Description
  Logic (formal meaning), Horn clauses with Forward and Backward Chaining
  (reasoning).

--- Roadmap of this demo ---
  Stage 1  Frames and script                          Student 1
  Stage 2  Semantic network                           Student 1
  Stage 3  First-Order Logic and Description Logic    Student 2
  Stage 4  Horn clause knowledge base                 Student 3
  Stage 5  Forward chaining                           Student 3
  Stage 6  Backward chaining and final decision       Student 4

  Scenario to run: S1: Brute force from an unknown device

[Press Enter for stage 1 (frames)]
```

Figure 7.3: Stage 0 - problem and motivation, with the roadmap of the demo

Rule representation (knowledge/rules.py). A rule is written as data. The head is a tuple, the body is a list of conditions, and “not” marks negation as failure.

```
Rule("R1_brute_force",
     ("brute_force", "?U"),
     [("failed_attempts", "?U", "?N"), (">=", "?N", 3)],
     stratum=0,
     doc="3 or more failed attempts means brute force"),

Rule("R5_high_risk_sensitive",
     ("high_risk", "?U", "?R"),
     [("suspicious_login", "?U"), ("requests", "?U", "?R"),
      ("sensitivity", "?R", "sensitive")],
     stratum=0,
     doc="suspicious user asking for a sensitive resource"),

Rule("R14_allow",
     ("decision", "?U", "?R", "allow"),
     [("requests", "?U", "?R"),
      ("not", ("suspicious_login", "?U")),
      ("not", ("unauthorized", "?U", "?R"))],
     stratum=2,
     doc="nothing suspicious and the user is authorized"),
```

Unification (engine/terms.py). Unification makes two terms equal by binding variables. The bindings are copied and never changed in place, so backtracking is safe.

```
def unify(x, y, b):
    x = walk(x, b)
    y = walk(y, b)
    if x == y:
        return b
    if is_var(x):
        return {**b, x: y}
    if is_var(y):
        return {**b, y: x}
    if isinstance(x, tuple) and isinstance(y, tuple) and len(x) == len(y):
        for a, c in zip(x, y):
            b = unify(a, c, b)
            if b is None:
                return None
        return b
    return None
```

Forward chaining (engine/forward_chaining.py). The loop runs stratum by stratum and repeats until no rule adds a new fact (fixpoint). Every new fact is saved in the trace together with the rule and the conditions that produced it.

```
def forward_chain(rules, initial_facts, strata=None):
    if strata is None:
        strata = group_by_stratum(rules)
    facts = dict.fromkeys(initial_facts)
    trace = []
    for group in strata:
        changed = True
        while changed:
            changed = False
            for rule in group:
```

```
                for bindings in match_conditions(rule.body, facts, {}):
                    new_fact = substitute(rule.head, bindings)
                    if new_fact not in facts:
                        facts[new_fact] = None
                        used = [substitute(c, bindings) for c in rule.body]
                        trace.append((rule.name, [fmt(u) for u in used],
                                      new_fact, used))
                        changed = True
    return facts, trace
```

Backward chaining (engine/backward_chaining.py, excerpt). A not goal succeeds when the negated goal cannot be proved. For other goals, the engine tries every rule whose head unifies with the goal, using fresh variable names.

```
if is_not(goal):                      # negation as failure
    target = substitute(goal[1], bindings)
    proved = any(True for _ in self.prove(target, bindings, depth + 1))
    if not proved:
        yield bindings
    return
...
for rule in self.rules:               # try each rule
    self.rename_count += 1
    fresh = rename_rule(rule, self.rename_count)
    new_bindings = unify(goal, fresh.head, bindings)
    if new_bindings is None:
        continue
    for final_bindings in self.prove_all(fresh.body, new_bindings, depth + 2):
        yield final_bindings          # all sub-goals proved
```

Final decision (engine/decision.py). If several decisions are derived for the same request, the one with the highest priority is chosen.

```
PRIORITY = ["deny", "flag", "allow"]

def final_decision(facts, user, resource):
    derived = {f[3] for f in facts
               if f[0] == "decision" and f[1] == user and f[2] == resource}
    for level in PRIORITY:
        if level in derived:
            return level
    return None
```

8. RESULTS AND DISCUSSION

8.1 Results

The complete output of scenario S1 is shown in the previous sections. Figure 8.1 shows the closing screen of the demo: the verdict, the decisions of both methods, the rules that fired in order, and a contrast with an ALLOW scenario.

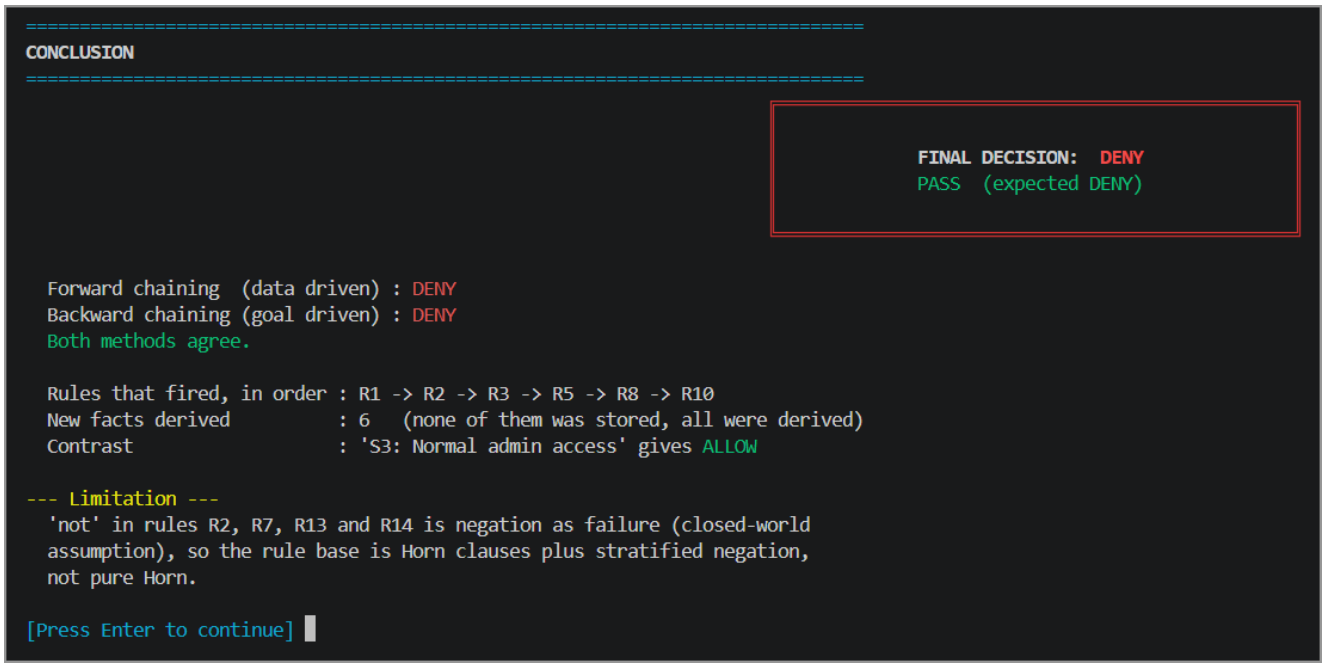


Figure 8.1: Conclusion screen of scenario S1

Seven scenarios were designed so that **every one of the 14 rules fires in at least one scenario**. Table 8.1 lists them with the rules that fired and the decision; the output of python main.py all is shown in Figure 8.2. All seven scenarios give the expected decision, and in each scenario forward and backward chaining agree.

Table 8.1: Test scenarios, rules fired and decisions

No	Scenario	Rules fired	Expected	Result
S1	ali fails the login 4 times from an unknown device and asks for the student database	R1, R2, R3, R5, R8, R10	DENY	PASS
S2	sara (faculty) logs in from an unknown device and opens a public page	R2, R3, R13	FLAG	PASS
S3	Admin hamza logs in from a trusted device and opens the student database	R14	ALLOW	PASS
S4	Student ali, trusted device, no failed attempts, asks for the admin panel	R7, R11	DENY	PASS
S5	Student ali asks for the admin panel and tries to change his own role	R7, R9, R11	DENY	PASS
S6	sara fails 5 times, then logs in from an unknown device to open a public page	R1, R2, R3, R8, R12	DENY	PASS
S7	ali fails 5 times from a trusted device and asks for the grades database	R1, R4, R6, R10	DENY	PASS

Two scenarios are interesting beyond the main case. In **S6** the requested page is harmless, but five failed attempts followed by an unknown device make the account look stolen, so rule R12 denies even a public

page. In **S7** the device is trusted, so only the failed attempts make the login suspicious (R4), and the critical resource makes the request high risk (R6).

```
PS G:\CyberGuard>
PS G:\CyberGuard> python main.py all

--- All scenarios ---
  #  Scenario                                  Expected  Got     Result  Facts derived
  1  S1: Brute force from an unknown device    DENY      DENY    PASS    6
  2  S2: Unknown device, public page           FLAG      FLAG    PASS    3
  3  S3: Normal admin access                   ALLOW     ALLOW   PASS    1
  4  S4: Student asks for an admin-only resource  DENY   DENY    PASS    2
  5  S5: Privilege escalation attempt          DENY      DENY    PASS    3
  6  S6: Account compromise on a public page   DENY      DENY    PASS    5
  7  S7: Brute force, trusted device, critical page  DENY  DENY  PASS    4

Scenarios passed: 7/7
```

Figure 8.2: Verification of all seven scenarios (python main.py all)

The what-if step changes input facts and runs the reasoning again with the same rules. In Figure 8.3, the failed attempts of ali are changed from 4 to 2 and the device dev7 is made trusted. Rule R1 no longer fires, dev7 is no longer unknown, so no suspicious login is derived and R14 allows the request: the decision changes from DENY to ALLOW. This shows that the decision is reasoned from the facts and is not hard-coded.

```
------------------------------------------------------------------------
LIVE STEP
------------------------------------------------------------------------
  [W]  What-if: change a fact and reason again
  [M]  Back to the menu
  [Q]  Quit

Choose: w
========================================================================
WHAT-IF: change a fact and run the reasoning again
========================================================================
What is this?
  The rules stay exactly the same. We only change one input fact and the
  engine derives the decision again from scratch.

Current situation: ali asks for student_db from dev7, failed attempts = 4, decision = DENY

New number of failed attempts (Enter keeps 4): 2
Make device dev7 trusted? y/n (Enter = no): y

--- Forward chaining on the changed facts ---
[FORWARD  1] R14_allow: requests(ali, student_db) + not suspicious_login(ali) + not unauthorized(ali, student_db)  =>  decision(ali, student_db, allow)

--- Result ---
Before: failed attempts = 4, device trusted = False  ->  DENY
After : failed attempts = 2, device trusted = True  ->  ALLOW
The decision changed because the facts changed.

Why:
  decision(ali, student_db, allow)   <= R14_allow
    requests(ali, student_db)   [given fact]
    not suspicious_login(ali)   [cannot be proved, so the 'not' holds]
    not unauthorized(ali, student_db)   [cannot be proved, so the 'not' holds]

------------------------------------------------------------------------
```

Figure 8.3: What-if analysis on scenario S1 (DENY changes to ALLOW)

The file tests/test_scenarios.py contains 15 automated tests (Python unittest), summarised in Table 8.2. All 15 pass (Figure 8.4).

Table 8.2: Automated tests

Group	Tests	What is checked
Scenario tests	4	Forward decision equals the expected decision; backward decision equals forward decision; every rule fires in some scenario; scenarios exist
Frame tests	5	Default reaches the engine; range facet rejects a bad value; values facet rejects a bad role; child frames inherit and override; demon fires for many failures
Logic tests	3	Subsumption follows the chain; disjointness is inherited; ABox conflict is detected
Engine tests	3	Trace is identical on every run; unsafe negation raises an error; what-if changes the decision

```
PS G:\CyberGuard> python tests/test_scenarios.py
test_trace_is_the_same_on_every_run (__main__.EngineTests.test_trace_is_the_same_on_every_run) ... ok
test_unsafe_negation_is_an_error (__main__.EngineTests.test_unsafe_negation_is_an_error) ... ok
test_what_if_changes_the_decision (__main__.EngineTests.test_what_if_changes_the_decision) ... ok
test_child_frame_inherits_and_overrides (__main__.FrameTests.test_child_frame_inherits_and_overrides) ... ok
test_default_reaches_the_engine (__main__.FrameTests.test_default_reaches_the_engine) ... ok
test_demon_fires_for_many_failures (__main__.FrameTests.test_demon_fires_for_many_failures) ... ok
test_range_facet_rejects_bad_value (__main__.FrameTests.test_range_facet_rejects_bad_value) ... ok
test_values_facet_rejects_bad_role (__main__.FrameTests.test_values_facet_rejects_bad_role) ... ok
test_abox_conflict_is_found (__main__.LogicTests.test_abox_conflict_is_found) ... ok
test_disjointness_is_inherited (__main__.LogicTests.test_disjointness_is_inherited) ... ok
test_subsumption_follows_the_chain (__main__.LogicTests.test_subsumption_follows_the_chain) ... ok
test_backward_decision_matches_forward (__main__.ScenarioTests.test_backward_decision_matches_forward) ... ok
test_every_rule_fires_in_some_scenario (__main__.ScenarioTests.test_every_rule_fires_in_some_scenario) ... ok
test_forward_decision_matches_expected (__main__.ScenarioTests.test_forward_decision_matches_expected) ... ok
test_there_are_scenarios (__main__.ScenarioTests.test_there_are_scenarios) ... ok

----------------------------------------------------------------------
Ran 15 tests in 0.058s

OK
PS G:\CyberGuard>
```

Figure 8.4: Output of the automated tests (15 tests, all ok)

8.2 Discussion

Objectives. The system meets its objectives. The domain is represented with frames and a semantic network, the policy is formalised in FOL and DL and converted into 14 Horn rules, and both chaining methods produce an ALLOW, FLAG or DENY decision with a reason chain. All seven scenarios give the expected decision, forward and backward chaining agree in every case, and all 15 automated tests pass.

Effectiveness of the KR approaches. Frames give defaults, facets and inheritance, and the semantic network gives an overview of how the concepts are connected. FOL is expressive enough to state every rule, and DL answers class-level questions (subsumption and disjointness) in a decidable way. Horn clauses make the policy executable. **Forward chaining** is data-driven: it derives every consequence of the facts and is the natural choice when all facts of a login are known. **Backward chaining** is goal-driven: it proves one hypothesis, such as “is deny justified?”, and looks only at the facts that matter. Both give the same decision, and both produce a trace that explains it.

Each formalism offers a different balance between what can be expressed and how efficiently conclusions can be computed. Table 8.3 summarises the limitations that we met in this domain.

Table 8.3: Limitations of the formalisms in the CyberGuard domain

Formalism	Strength	Limitation in this domain
Propositional logic	Simple and decidable	No variables. One rule per user and resource would be needed (brute_force_ali, brute_force_sara, ...). It cannot say “for every user”.
First-Order Logic	Variables, quantifiers and relations; can state every rule	Entailment is only semi-decidable: a proof may never finish for a false goal.
Description Logic	Decidable; good for classes (subsumption, disjointness)	Cannot express rules that join several facts, such as R5 (user + resource + risk). Such rules are executed as Horn clauses.
Horn clauses	Only one positive literal per rule; fast (polynomial forward chaining without function symbols)	Cannot express disjunction or classical negation. We use negation as failure, which assumes that what is not known is false.

How the design resolves the trade-off. FOL is used as the specification language because it is the most expressive. The executable knowledge base is restricted to Horn clauses with stratified negation as failure. Because the facts are finite and the rules contain no function symbols, forward chaining always terminates. The rules are also not recursive, so backward chaining always terminates as well. Description Logic is used only for class-level reasoning (subsumption and disjointness), where it is decidable. The price of this choice is the closed-world assumption: a device that is not in the trusted list is treated as unknown, and a user who is not proved suspicious is treated as not suspicious.

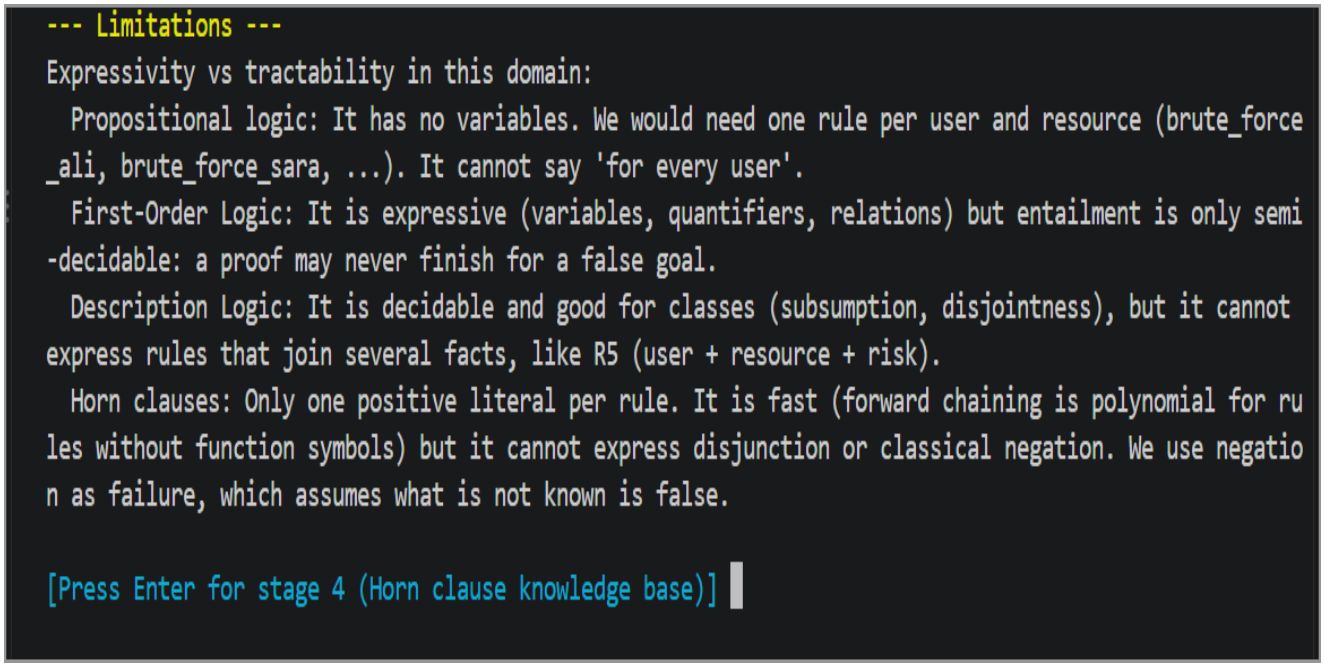


Figure 8.5: Limitations printed by the program (Stage 3)

Limitations. The main limitations are the closed-world assumption behind negation as failure, the absence of disjunctive reasoning in Horn clauses, and the small, hand-written knowledge base of four users, four devices and four resources.

9. CONCLUSION

9.1 Conclusion and Future Improvements

CyberGuard shows how several knowledge representation paradigms work together on one real-world problem. Frames and the semantic network structure the domain; First-Order Logic and Description Logic give the policy a formal meaning; Horn clauses turn the policy into executable rules; and forward and backward chaining derive decisions with a complete explanation. Seven scenarios cover all 14 rules, all of them give the expected decision, forward and backward chaining agree in every case, and 15 automated tests pass.

Possible improvements and extensions are reading real login logs from a database, adding time-based rules (for example failed attempts within ten minutes), using a full Description Logic reasoner for the class hierarchy, and adding probabilistic risk scores next to the logical rules.

10. REFERENCES

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[2] R. J. Brachman and H. J. Levesque, Knowledge Representation and Reasoning. San Francisco, CA, USA: Morgan Kaufmann, 2004.

[3] Python Software Foundation, “Python 3 documentation.” [Online]. Available: https://docs.python.org/3/ (accessed Nov. 3, 2026).

[4] J. D. Hunter, “Matplotlib: A 2D graphics environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.

[5] NED University of Engineering & Technology, CT-351 Knowledge Representation & Reasoning, course lecture notes and CCP report template, Department of Computer Science and IT, 2026.

11. TEAM PARTICIPATION

The project was carried out by a team of four students. The work was divided as follows, and each member presents and explains the part marked below during the viva. All members took part in the design discussions, in the testing of the scenarios and in the preparation of this report, and every member is able to explain every stage of the complete system.

Student Name	Roll Number	Contribution / Responsibilities
Umair Hasan	AI-23018	Backward chaining, decision logic, testing, report integration and references (Sections 6.2, 7.2, 8.1 and 10)
Abrar Shakeel Ghoury	AI-23019	Domain analysis, frames, script and frame instances, semantic network, and all diagrams (Sections 2, 3 and 7.1)
Amna Amjad	AI-23031	Introduction, Horn clause knowledge base and forward chaining (Sections 1, 5 and 6.1)
Mirza Abdul Raffay Baig	AI-23033	First-Order Logic, Description Logic axioms, limitations analysis and conclusion (Sections 4, 8.2 and 9)

Declaration: We confirm that the submitted report represents our group's work and that each member has contributed as stated above.

Group Representative: _____ **Date:** _____